materia: Frontend tipo: guia fecha: 2026-11-13 probado: 2026-11-13 tags: [clase, frontend, live-coding]

Componentes del catálogo · Guía de la clase del 13-nov-2026

Programación IV · Frontend · UNIAJC · 3 horas

Volver a [[2026-11-13 - Componentes del catalogo]] · [[Frontend]]

Hoy el catálogo queda igual al mockup, con **datos de prueba** que tienen la misma forma que los que vamos a traer de la API. Todavía no hay JavaScript en el navegador: los filtros, el buscador y la paginación se ven, pero no funcionan. Eso llega después.

Probado hoy sobre el repositorio: `npm run build` genera **6 páginas**, **0 archivos `.js`**, y a 320 px las seis responden `true`.

1. Qué páginas se desarrollan hoy

Página	Dirección	Qué cambia hoy
Catálogo	`/catalogo`	Pasa de "se construye en la próxima clase" a la pantalla completa: filtros, buscador, conteo, seis tarjetas y paginación
Inicio	`/`	El recuadro punteado de "destacadas" se reemplaza por tres tarjetas reales
Estados (nueva)	`/estados`	Página de referencia con los cuatro estados: cargando, con datos, vacío y error

Componentes nuevos: `Card`, `CardGrid`, `BarraDeBusqueda`, `Filtros`, `Paginacion` (en `components/catalogo/`), `Estado` y `Esqueleto` (en `components/ui/`). Datos: `src/data/datos-de-prueba.js`.

Mockups de referencia: `docs/04-mockup/catalogo-escritorio.png`, `catalogo-movil.png`, `estados-escritorio.png`.

2. Estado del repositorio al empezar

- `main` tiene el taller de ayer terminado: Layout, Header, Footer, Boton, Etiqueta, portada y las cinco rutas. Se corrigieron también los errores del live coding (ver [[Taller 2026-11-12 resuelto]]).
- La solución de hoy está en la rama **`catalogo-2026-11-13`**. Si algo se rompe en vivo: `git switch catalogo-2026-11-13` para mostrar cómo debe quedar, y `git switch main` para volver.
- El código de cada archivo está en `Frontend_programacion_IV/Clase-2026-11-13-Catalogo/codigo-live-coding/`.

3. Distribución del tiempo

Minuto	Qué se hace
0-15	Repaso del taller de ayer con las diapositivas · `git pull` · sitio publicado al lado del mockup
15-25	Paso 1 · datos de prueba
25-50	Paso 2 · `Card.astro` · error intencional sin `width` y `height`
50-65	Paso 3 · `CardGrid.astro` y destacadas en la portada
65-75	Descanso (commit antes de salir)
75-100	Pasos 4 y 5 · buscador, filtros, paginación y página del catálogo · error intencional sin `min-width: 0`
100-120	Pasos 6 y 7 · `Estado`, `Esqueleto` y página `/estados`
120-130	Paso 8 · prueba a 320 px, `build`, `push`
130-170	Taller en parejas
170-180	Cierre

4. Cómo copiar el código más rápido en vivo

En el repositorio quedó el archivo `.vscode/live-coding.code-snippets`. VS Code lo carga solo al abrir la carpeta. En un archivo vacío escribe el prefijo y presiona **Tab**:

Prefijo	Qué pega
`lc2-card`	El marcado de la tarjeta, sin estilos
`lc2-card-css`	Solo el bloque `<style>` de la tarjeta
`lc2-card-todo`	El archivo completo (plan B si vas atrasado)

Lo mismo para `lc1-datos-todo`, `lc3-grid`, `lc4-busqueda`, `lc4-filtros`, `lc4-paginacion`, `lc5-catalogo`, `lc6-estado`, `lc6-esqueleto` y `lc7-estados`, cada uno con su `-css` y su `-todo`.

La regla para no perder tiempo: escribe a mano lo que se está enseñando (las props, el `map`, la línea del Grid, el `::after`) y pega con el fragmento lo que es repetitivo (el bloque de estilos, los seis objetos de datos). Nadie aprende escribiendo `border-radius` por décima vez.

Otros atajos de VS Code que ayudan:

- **Pantalla dividida:** `Ctrl+\` con esta guía a la izquierda y el archivo a la derecha.
- **Varios cursores:** `Alt+clic`, o `Ctrl+D` para seleccionar la siguiente coincidencia y cambiar varias a la vez.
- **Emmet:** `ul.rejilla>li` + Tab escribe la estructura.
- **Formatear:** `Shift+Alt+F` arregla la sangría de lo pegado.
- **Abrir archivo:** `Ctrl+P` y el nombre, sin buscar en el árbol.
- **Terminal:** `Ctrl+ñ` (o `Ctrl+``) abre y cierra la terminal.
- **Si te atrasas:** pega el `-todo` del paso y explica leyendo. Es mejor terminar la clase que escribir todo.

Paso 1 (15-25) · Datos de prueba

Crea `src/data/datos-de-prueba.js` (fragmento `lc1-datos-todo`):

`src/data/datos-de-prueba.js`

```
// datos-de-prueba.js
// Seis recetas falsas con los MISMOS campos que vamos a sacar de la API.
// Si la forma de los datos no cambia, los componentes tampoco cambian
// cuando lleguen los datos reales.

export const recetas = [
  {
    id: "52772",
    nombre: "Pollo teriyaki al horno",
    categoria: "Pollo",
    area: "Japonesa",
    imagen: "img/recetas/pollo-teriyaki.svg",
    minutos: 45,
    dificultad: "Media",
  },
  {
    id: "52771",
    nombre: "Pasta con tomate y albahaca",
    categoria: "Pasta",
    area: "Italiana",
    imagen: "img/recetas/pasta-tomate.svg",
    minutos: 25,
    dificultad: "Facil",
  },
  {
    id: "52968",
    nombre: "Arroz con huevo y verduras",
    categoria: "Vegetariana",
    area: "Asiatica",
    imagen: "img/recetas/arroz-verduras.svg",
    minutos: 20,
    dificultad: "Facil",
  },
  {
    id: "52959",
    nombre: "Tortilla de papa",
    categoria: "Desayuno",
    area: "Espanola",
    imagen: "img/recetas/tortilla-papa.svg",
    minutos: 35,
    dificultad: "Media",
  },
  {
    id: "52785",
    nombre: "Sopa de lentejas",
    categoria: "Sopa",
    area: "Colombiana",
    imagen: "img/recetas/sopa-lentejas.svg",
    minutos: 50,
    dificultad: "Facil",
  },
  {
    id: "53013",
    nombre: "Ensalada de garbanzos",
    categoria: "Vegetariana",
    area: "Mediterranea",
    imagen: "img/recetas/ensalada-garbanzos.svg",
```

```
    minutos: 15,
    dificultad: "Facil",
  },
];

// Listas para los filtros, calculadas a partir de los datos.
// new Set() quita los repetidos: "Vegetariana" aparece dos veces y queda una.
export const categorias = [...new Set(recetas.map((r) => r.categoria))];
export const dificultades = [...new Set(recetas.map((r) => r.dificultad))];
```

> Qué decir: "Antes de conectar la API diseñamos con datos falsos, pero con la misma forma: `id`, `nombre`, `categoria`, `area`, `imagen`, `minutos`, `dificultad`. Cuando lleguen los datos reales, una función los va a traducir a esta forma y las tarjetas no se enteran del cambio."

Puntos para explicar:

- **`export const`**: el archivo exporta tres cosas y cada página importa solo las que necesita.
- **`new Set()`** quita los repetidos: "Vegetariana" está dos veces y en los filtros aparece una.
- Copia también las seis ilustraciones de `public/img/recetas/` desde `codigo-live-coding/`.

Paso 2 (25-50) · `Card.astro`: la tarjeta de una receta

Crea `src/components/catalogo/Card.astro`. Escribe a mano el marcado (o `lc2-card`) y pega los estilos con `lc2-card-css`:

`src/components/catalogo/Card.astro`

```
---
// Card.astro
// Resumen de una receta. Recibe la receta por props: no busca datos.
import Etiqueta from "../ui/Etiqueta.astro";

const { receta } = Astro.props;
const { id, nombre, categoria, area, imagen, minutos, dificultad } = receta;
const base = import.meta.env.BASE_URL;
---

<article class="tarjeta">
  <div class="tarjeta__medio">
    <!-- width y height reservan el espacio antes de que llegue la imagen -->
    <img
      class="tarjeta__imagen"
      src={`${base}${imagen}`}
      alt=""
      width="400"
      height="300"
      loading="lazy"
    />
    <span class="tarjeta__tiempo">{minutos} min</span>
    <button
      class="tarjeta__favorito"
      type="button"
      aria-label={`Guardar ${nombre} en favoritos`}
      aria-pressed="false"
    >
      <svg viewBox="0 0 24 24" width="18" height="18" fill="none" stroke="currentColor"
        stroke-width="2" stroke-linecap="round" stroke-linejoin="round" aria-hidden="true">
        <path d="M20.8 4.6a5.5 5.5 0 0 0-7.8 0L12 5.7l-1-1.1a5.5 5.5 0 0 0-7.8 7.8L12 21.2l8.8-8.8a5.5
5.5 0 0 0 0-7.8Z" />
      </svg>
    </button>
  </div>

  <div class="tarjeta__cuerpo">
    <h3 class="tarjeta__titulo">
      <a class="tarjeta__enlace" href={`${base}catalogo/${id}`}>{nombre}</a>
    </h3>
    <p class="tarjeta__meta">{categoria} · {area}</p>
    <div class="tarjeta__etiquetas">
      <Etiqueta>{dificultad}</Etiqueta>
      <Etiqueta>{minutos} min aprox.</Etiqueta>
    </div>
  </div>
</article>

<style>
  .tarjeta {
    position: relative; /* el ::after del enlace se mide desde aqui */
    display: flex;
    flex-direction: column;
    height: 100%;
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio-m);
```

```
  overflow: hidden;
  box-shadow: var(--sombra-1);
  transition: box-shadow var(--transicion), transform var(--transicion);
}

.tarjeta:hover {
  box-shadow: var(--sombra-2);
  transform: translateY(-2px);
}

.tarjeta__medio {
  position: relative;
}

.tarjeta__imagen {
  width: 100%;
  height: auto;
  aspect-ratio: 4 / 3;
  object-fit: cover;
  background: var(--color-superficie);
}

.tarjeta__tiempo {
  position: absolute;
  left: var(--esp-2);
  bottom: var(--esp-2);
  padding: 0.3rem 0.6rem;
  border-radius: var(--radio-pildora);
  background: rgba(36, 27, 22, 0.78);
  color: #ffffff;
  font-size: var(--texto-xs);
  font-weight: 700;
}

.tarjeta__favorito {
  position: absolute;
  top: var(--esp-2);
  right: var(--esp-2);
  z-index: 2; /* por encima del enlace que cubre la tarjeta */
  display: grid;
  place-items: center;
  width: 44px;
  height: 44px;
  border: 1px solid var(--color-borde);
  border-radius: var(--radio-pildora);
  background: var(--color-fondo);
  color: var(--color-texto);
  cursor: pointer;
}

.tarjeta__favorito:hover {
  color: var(--color-marca);
}

.tarjeta__cuerpo {
  display: flex;
  flex-direction: column;
```

```
    gap: var(--esp-2);
    padding: var(--esp-3);
  }

  .tarjeta__titulo {
    font-size: var(--texto-m);
    margin: 0;
  }

  .tarjeta__enlace {
    color: var(--color-texto);
    text-decoration: none;
  }

  /* Toda la tarjeta es clicable: el ::after del enlace la cubre completa */
  .tarjeta__enlace::after {
    content: "";
    position: absolute;
    inset: 0;
    z-index: 1;
  }

  .tarjeta__enlace:focus-visible {
    outline: none;
  }

  .tarjeta:has(.tarjeta__enlace:focus-visible) {
    outline: 3px solid var(--color-marca);
    outline-offset: 2px;
  }

  .tarjeta__meta {
    margin: 0;
    font-size: var(--texto-s);
    color: var(--color-texto-suave);
  }

  .tarjeta__etiquetas {
    display: flex;
    flex-wrap: wrap;
    gap: var(--esp-1);
  }
</style>
```

> Qué decir: "Una tarjeta recibe una receta y la dibuja. No va a buscar datos: eso lo decide la página. Por eso el mismo componente sirve en la portada, en el catálogo y en favoritos."

Explica cuatro decisiones, señalándolas en el mockup:

1. **El tiempo va sobre la imagen**, porque es el dato que decide si se entra o no (briefing).
2. **El favorito mide 44 × 44 px**: es el tamaño que un dedo acierta. Es un `<button>`, porque hace algo en la misma página; `aria-pressed` dice si está marcado.

3. **Toda la tarjeta es clicable**, pero en el HTML hay **un solo enlace**: el del título. Su `::after` con `position: absolute; inset: 0` lo estira sobre la tarjeta (que tiene `position: relative`). El botón de favorito queda encima con `z-index: 2`.
4. `alt=""` en la imagen: es decorativa, el nombre ya está en el título. Un lector de pantalla no tiene que leerlo dos veces.

Error intencional (CLS): borra `width="400"` y `height="300"` de la imagen. En DevTools → Network → **Slow 3G** recarga: el texto aparece arriba y salta hacia abajo cuando llega la imagen. Vuelve a poner los dos atributos: el navegador reserva el espacio antes de descargarla.

> El enlace del título lleva a `catalogo/52772`, una página que todavía no existe (da 404). Se crea cuando conectemos la API, con una página por receta. Dilo antes de que alguien haga clic.

Paso 3 (50-65) · `CardGrid.astro` y las destacadas de la portada

Crea `src/components/catalogo/CardGrid.astro` (`lc3-grid`):

`src/components/catalogo/CardGrid.astro`

```
---
// CardGrid.astro
// Rejilla de tarjetas. Una sola linea de Grid reemplaza tres media queries.
import Card from "./Card.astro";

const { recetas } = Astro.props;
---

<ul class="rejilla">
  {recetas.map((receta) => (
    <li>
      <Card receta={receta} />
    </li>
  ))}
</ul>

<style>
  .rejilla {
    display: grid;
    /* Caben tantas columnas de 260 px como permita el ancho; lo que sobra se reparte */
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: var(--esp-4);
    list-style: none;
    margin: 0;
    padding: 0;
  }
</style>
```

> Qué decir: "Ayer, para pasar de una a tres columnas, escribimos media queries. Esta línea lo hace sola: caben tantas columnas de 260 px como permita el ancho, y lo que sobra se reparte. En el celular cabe una,

> en una tablet dos y en escritorio tres. Sin un solo `@media`."

Compáralo en la pizarra con lo que habría que escribir sin ella:

```
.rejilla { grid-template-columns: 1fr; }
@media (min-width: 600px)  { .rejilla { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1000px) { .rejilla { grid-template-columns: repeat(3, 1fr); } }
```

`auto-fit` o `auto-fill`: con `auto-fill` quedan columnas vacías si hay pocas tarjetas (en la portada, con tres, sobra un hueco a la derecha). Con `auto-fit` las tres se estiran y ocupan todo. Cámbialo en vivo y muestra la diferencia.

Ahora usa la rejilla en la portada. En `src/pages/index.astro`, agrega los dos `import`:

```
import CardGrid from "../components/catalogo/CardGrid.astro";
import { recetas } from "../data/datos-de-prueba.js";
```

y reemplaza el recuadro punteado `<p class="pendiente">…</p>` por:

```
<CardGrid recetas={recetas.slice(0, 3)} />
```

> Qué decir: "`slice(0, 3)` toma las tres primeras. Los datos bajan así: la página los importa, se los pasa a `CardGrid`, y `CardGrid` le pasa una receta a cada `Card`. Cada componente recibe solo lo que necesita."

Descanso: `git add .` y `git commit -m "feat: tarjeta y rejilla con datos de prueba"`.

Paso 4 (75-90) · Buscador, filtros y paginación (se ven, todavía no funcionan)

Tres componentes en `src/components/catalogo/`. Escribe el marcado y pega los estilos.

`BarraDeBusqueda.astro` (`lc4-busqueda`):

`src/components/catalogo/BarraDeBusqueda.astro`

```
---
// BarraDeBusqueda.astro
// Hoy solo se ve. Empieza a buscar cuando agreguemos JavaScript.
const { placeholder = "Buscar receta por nombre o ingrediente" } = Astro.props;
---

<form class="busqueda" role="search" onsubmit="return false">
  <label class="solo-lectores" for="buscar">Buscar recetas</label>
  <svg class="busqueda__icono" viewBox="0 0 24 24" width="18" height="18" fill="none"
    stroke="currentColor" stroke-width="2" stroke-linecap="round" aria-hidden="true">
    <circle cx="11" cy="11" r="7" />
    <path d="m20 20-3.5-3.5" />
  </svg>
  <input class="busqueda__campo" id="buscar" type="search" placeholder={placeholder} />
</form>

<style>
  .busqueda {
    position: relative;
  }

  .busqueda__icono {
    position: absolute;
    left: var(--esp-3);
    top: 50%;
    transform: translateY(-50%);
    color: var(--color-texto-tenue);
    pointer-events: none;
  }

  .busqueda__campo {
    width: 100%;
    min-height: 48px;
    padding: 0 var(--esp-3) 0 calc(var(--esp-3) + 28px);
    font: inherit;
    color: var(--color-texto);
    background: var(--color-fondo);
    border: 1px solid var(--color-borde-fuerte);
    border-radius: var(--radio-s);
  }

  .busqueda__campo::placeholder {
    color: var(--color-texto-tenue);
  }
</style>
```

- El `<label>` existe aunque no se vea (`solo-lectores`): sin él, un lector de pantalla anuncia "campo de edición" y nada más. El `placeholder` no reemplaza al `label`.

`Filtros.astro` (`lc4-filtros`):

`src/components/catalogo/Filtros.astro`

```
---
// Filtros.astro
// Una fila desplazable en el celular; una barra lateral en escritorio.
// Hoy solo se ven. Empiezan a filtrar cuando agreguemos JavaScript.
const { categorias = [], dificultades = [] } = Astro.props;
---

<aside class="filtros" aria-label="Filtros del catalogo">
  <div class="filtros__grupo">
    <p class="filtros__titulo">Categoria</p>
    <ul class="filtros__lista">
      <li><button class="filtro" type="button" aria-pressed="true">Todas</button></li>
      {categorias.map((c) => (
        <li><button class="filtro" type="button" aria-pressed="false">{c}</button></li>
      ))}
    </ul>
  </div>

  <div class="filtros__grupo">
    <p class="filtros__titulo">Dificultad</p>
    <ul class="filtros__lista">
      {dificultades.map((d) => (
        <li><button class="filtro" type="button" aria-pressed="false">{d}</button></li>
      ))}
    </ul>
  </div>
</aside>

<style>
  .filtros {
    display: flex;
    flex-direction: column;
    gap: var(--esp-3);
    min-width: 0; /* sin esto, la fila larga empuja la pagina hacia el lado */
  }

  .filtros__titulo {
    margin: 0 0 var(--esp-2);
    font-size: var(--texto-xs);
    letter-spacing: 0.1em;
    text-transform: uppercase;
    font-weight: 700;
    color: var(--color-texto-tenue);
  }

  /* Celular: una fila que se desplaza de lado */
  .filtros__lista {
    display: flex;
    gap: var(--esp-2);
    list-style: none;
    margin: 0;
    padding: 0 0 var(--esp-1);
    overflow-x: auto;
  }

  .filtro {
    min-height: 40px;
```

```
    padding: 0 var(--esp-3);
    font: inherit;
    font-size: var(--texto-s);
    font-weight: 600;
    white-space: nowrap;
    color: var(--color-texto-suave);
    background: var(--color-fondo);
    border: 1px solid var(--color-borde-fuerte);
    border-radius: var(--radio-pildora);
    cursor: pointer;
  }

  .filtro:hover {
    border-color: var(--color-marca);
    color: var(--color-marca);
  }

  .filtro[aria-pressed="true"] {
    background: var(--color-marca);
    border-color: var(--color-marca);
    color: #ffffff;
  }

  /* Escritorio: tarjeta lateral y botones uno debajo del otro */
  @media (min-width: 900px) {
    .filtros {
      gap: var(--esp-5);
      padding: var(--esp-4);
      background: var(--color-fondo);
      border: 1px solid var(--color-borde);
      border-radius: var(--radio-m);
    }

    .filtros__lista {
      flex-direction: column;
      align-items: flex-start;
      overflow: visible;
      padding: 0;
    }
  }
</style>
```

- Los filtros son **botones**, no enlaces: cambian lo que se ve en la misma página. `aria-pressed="true"` marca el activo, y el CSS lo pinta con `[aria-pressed="true"]`. El mismo atributo sirve para la accesibilidad y para el estilo.
- **Móvil primero:** en el celular cada grupo es una fila que se desplaza de lado. Desde 900 px se vuelve una tarjeta lateral con los botones en columna.

`Paginacion.astro` (`lc4-paginacion`):

`src/components/catalogo/Paginacion.astro`

```
---
// Paginacion.astro
// Hoy solo se ve. Cambia de pagina cuando agreguemos JavaScript.
const { actual = 1, total = 3 } = Astro.props;
const paginas = Array.from({ length: total }, (_, i) => i + 1);
---

<nav class="paginacion" aria-label="Paginas del catalogo">
  <button class="paginacion__boton" type="button" disabled={actual === 1}>Anterior</button>
  <ol class="paginacion__lista">
    {paginas.map((n) => (
      <li>
        <button
          class="paginacion__boton paginacion__numero"
          type="button"
          aria-current={n === actual ? "page" : undefined}
          aria-label={`Pagina ${n}`}
        >
          {n}
        </button>
      </li>
    ))}
  </ol>
  <button class="paginacion__boton" type="button" disabled={actual === total}>Siguiente</button>
</nav>

<style>
  .paginacion {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    align-items: center;
    gap: var(--esp-2);
    margin-top: var(--esp-6);
  }

  .paginacion__lista {
    display: flex;
    gap: var(--esp-1);
    list-style: none;
    margin: 0;
    padding: 0;
  }

  .paginacion__boton {
    min-width: 44px;
    min-height: 44px;
    padding: 0 var(--esp-3);
    font: inherit;
    font-weight: 600;
    color: var(--color-texto);
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio-s);
    cursor: pointer;
  }
```

```
  .paginacion__numero {
    padding: 0;
  }

  .paginacion__boton:hover:not(:disabled) {
    border-color: var(--color-marca);
    color: var(--color-marca);
  }

  .paginacion__boton[aria-current="page"] {
    background: var(--color-marca);
    border-color: var(--color-marca);
    color: #ffffff;
  }

  .paginacion__boton:disabled {
    color: var(--color-texto-tenue);
    background: var(--color-lienzo);
    cursor: not-allowed;
  }
</style>
```

- `Array.from({ length: total }, (_, i) => i + 1)` crea `[1, 2, 3]` para hacer el `map`.
- `aria-current="page"` en la página actual, igual que en el Header. `disabled` en "Anterior" porque estamos en la primera.

Paso 5 (90-100) · La página del catálogo

Reemplaza `src/pages/catalogo.astro` (`lc5-catalogo` y `lc5-catalogo-css`):

`src/pages/catalogo.astro`

```
---
import Layout from "../layouts/Layout.astro";
import Filtros from "../components/catalogo/Filtros.astro";
import BarraDeBusqueda from "../components/catalogo/BarraDeBusqueda.astro";
import CardGrid from "../components/catalogo/CardGrid.astro";
import Paginacion from "../components/catalogo/Paginacion.astro";
import { recetas, categorias, dificultades } from "../data/datos-de-prueba.js";
---

<Layout
  titulo="Catalogo — SaborU"
  descripcion="Catalogo de recetas rapidas con tiempo y dificultad visibles."
  activo="catalogo"
>
  <div class="contenedor">
    <p class="rotulo">Catalogo</p>
    <h1>Que puedes cocinar hoy</h1>
    <p class="entrada">
      Filtra por categoria o por dificultad. Cada ficha indica cuanto tarda de
      verdad, no cuanto dura el video.
    </p>

    <div class="catalogo">
      <Filtros categorias={categorias} dificultades={dificultades} />

      <div class="catalogo__resultados">
        <BarraDeBusqueda />
        <p class="catalogo__conteo" role="status">
          <strong>{recetas.length}</strong> recetas encontradas
        </p>
        <CardGrid recetas={recetas} />
        <Paginacion actual={1} total={3} />
      </div>
    </div>
  </div>
</Layout>

<style>
  .entrada {
    max-width: var(--ancho-lectura);
    color: var(--color-texto-suave);
  }

  /* Celular: filtros arriba y resultados debajo (una sola columna) */
  .catalogo {
    display: grid;
    gap: var(--esp-4);
    margin-top: var(--esp-5);
  }

  /* Escritorio: barra lateral de 240 px y el resto para las tarjetas */
  @media (min-width: 900px) {
    .catalogo {
      grid-template-columns: 240px 1fr;
      gap: var(--esp-5);
      align-items: start;
    }
```

```
  }

  .catalogo__resultados {
    min-width: 0; /* un hijo de Grid no se encoge por debajo de su contenido sin esto */
  }

  .catalogo__conteo {
    margin: var(--esp-3) 0;
    font-size: var(--texto-s);
    color: var(--color-texto-suave);
  }

  .catalogo__conteo strong {
    color: var(--color-texto);
  }
</style>
```

> Qué decir: "La página no dibuja nada por su cuenta: importa datos y componentes, y los acomoda. En el celular, filtros arriba y tarjetas abajo; desde 900 px, una barra lateral de 240 px y el resto para las tarjetas."

Error intencional (`min-width: 0`): borra `min-width: 0` de `.catalogo__resultados` y también de `.filtros` en `Filtros.astro`. Pon el simulador a 320 px y ejecuta en la consola:

```
document.documentElement.scrollWidth === document.documentElement.clientWidth
```

Responde `false`: la fila de filtros, que es más ancha que la pantalla, empuja la columna y la página se desplaza de lado. Un hijo de Grid o Flexbox no se encoge por debajo de su contenido, a menos que le digamos `min-width: 0`. Vuelve a ponerlo: `true`.

Paso 6 (100-110) · `Estado.astro` y `Esqueleto.astro`

En `src/components/ui/`.

`Estado.astro` (`lc6-estado`): vacío y error son **el mismo componente**, con distinto icono, color y textos. La acción entra por `<slot />`, así cada pantalla decide su botón.

`src/components/ui/Estado.astro`

```
---
// Estado.astro
// "Vacio" y "Error" comparten la misma estructura: icono, titulo,
// explicacion y una salida. Solo cambian el icono, el color y los textos.
const { tipo = "vacio", titulo, mensaje } = Astro.props;
const esError = tipo === "error";
---

<div class:list={["estado", { "estado--error": esError }]} role={esError ? "alert" : "status"}>
  <span class="estado__icono" aria-hidden="true">
    {esError ? (
      <svg viewBox="0 0 24 24" width="22" height="22" fill="none" stroke="currentColor" stroke-
width="2" stroke-linecap="round">
        <circle cx="12" cy="12" r="9" />
        <path d="M12 7.5v5.5M12 16.5v.01" />
      </svg>
    ) : (
      <svg viewBox="0 0 24 24" width="22" height="22" fill="none" stroke="currentColor" stroke-
width="1.9" stroke-linecap="round">
        <path d="M4 13h16" />
        <path d="M6 13a6 6 0 0 0 0 12 0" />
        <path d="M9 7.5C9 6 10.3 5 12 5s3 1 3 2.5" />
      </svg>
    )}
  </span>
  <h3 class="estado__titulo">{titulo}</h3>
  <p class="estado__mensaje">{mensaje}</p>
  <div class="estado__accion">
    <slot />
  </div>
</div>

<style>
  .estado {
    display: flex;
    flex-direction: column;
    align-items: center;
    text-align: center;
    padding: var(--esp-6) var(--esp-4);
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio-m);
  }

  .estado__icono {
    display: grid;
    place-items: center;
    width: 52px;
    height: 52px;
    margin-bottom: var(--esp-3);
    border-radius: var(--radio-pildora);
    background: var(--color-superficie);
    color: var(--color-texto-suave);
  }

  .estado--error .estado__icono {
    background: var(--color-error-suave);
```

```
    color: var(--color-error);
  }

  .estado__titulo {
    margin: 0 0 var(--esp-2);
  }

  .estado__mensaje {
    max-width: 46ch;
    margin: 0 0 var(--esp-4);
    color: var(--color-texto-suave);
  }
</style>
```

- `role="status"` para el vacío y `role="alert"` para el error: el lector de pantalla los anuncia sin que el usuario tenga que buscarlos.
- El rojo no es la única señal del error: también cambia el icono y el texto.

`Esqueleto.astro` (`lc6-esqueleto`): bloques con la forma de la tarjeta, no una rueda girando.

`src/components/ui/Esqueleto.astro`

```
---
// Esqueleto.astro
// Estado "cargando": bloques con la forma de la tarjeta que va a llegar,
// no una rueda girando sobre una pantalla en blanco.
const { cantidad = 3 } = Astro.props;
const bloques = Array.from({ length: cantidad });
---

<ul class="esqueletos" aria-busy="true" aria-label="Cargando recetas">
  {bloques.map(() => (
    <li class="esqueleto">
      <div class="esqueleto__imagen"></div>
      <div class="esqueleto__cuerpo">
        <div class="esqueleto__linea esqueleto__linea--titulo"></div>
        <div class="esqueleto__linea"></div>
        <div class="esqueleto__linea esqueleto__linea--corta"></div>
      </div>
    </li>
  ))}
</ul>

<style>
  .esqueletos {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: var(--esp-4);
    list-style: none;
    margin: 0;
    padding: 0;
  }

  .esqueleto {
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio-m);
    overflow: hidden;
  }

  .esqueleto__imagen,
  .esqueleto__linea {
    background: var(--color-superficie);
    animation: pulso 1.4s ease-in-out infinite;
  }

  .esqueleto__imagen {
    aspect-ratio: 4 / 3;
  }

  .esqueleto__cuerpo {
    display: flex;
    flex-direction: column;
    gap: var(--esp-2);
    padding: var(--esp-3);
  }

  .esqueleto__linea {
    height: 10px;
```

```
    border-radius: var(--radio-pildora);
  }

  .esqueleto__linea--titulo {
    width: 70%;
    height: 14px;
  }

  .esqueleto__linea--corta {
    width: 40%;
  }

  @keyframes pulso {
    50% {
      opacity: 0.5;
    }
  }
</style>
```

- La animación respeta `prefers-reduced-motion`: la regla ya está en `global.css` desde ayer.

Paso 7 (110-120) · La página `/estados`

Crea `src/pages/estados.astro` (`lc7-estados`):

`src/pages/estados.astro`

```
---
import Layout from "../layouts/Layout.astro";
import CardGrid from "../components/catalogo/CardGrid.astro";
import Esqueleto from "../components/ui/Esqueleto.astro";
import Estado from "../components/ui/Estado.astro";
import Boton from "../components/ui/Boton.astro";
import { recetas } from "../data/datos-de-prueba.js";

const base = import.meta.env.BASE_URL;
---

<Layout
  titulo="Los cuatro estados — SaborU"
  descripcion="Referencia de interfaz: cargando, con datos, vacio y error."
  activo="catalogo"
>
  <div class="contenedor">
    <p class="rotulo">Referencia de interfaz</p>
    <h1>Los cuatro estados</h1>
    <p class="entrada">
      Toda pantalla que muestra datos existe en cuatro versiones. Disenar solo
      la segunda es disenar la cuarta parte de la pantalla.
    </p>

    <section class="bloque">
      <h2>1. Cargando</h2>
      <p class="bloque__ayuda">Bloques con la forma del contenido que va a llegar.</p>
      <Esqueleto cantidad={3} />
    </section>

    <section class="bloque">
      <h2>2. Con datos</h2>
      <p class="bloque__ayuda">El caso que todo el mundo disena.</p>
      <CardGrid recetas={recetas.slice(0, 3)} />
    </section>

    <section class="bloque">
      <h2>3. Vacio</h2>
      <p class="bloque__ayuda">Dice que paso, por que, y ofrece una salida.</p>
      <Estado
        tipo="vacio"
        titulo="No encontramos recetas con ese filtro"
        mensaje="Prueba con menos filtros o busca por un ingrediente que tengas en la nevera."
      >
        <Boton href={`${base}catalogo`}>Ver todas las recetas</Boton>
      </Estado>
    </section>

    <section class="bloque">
      <h2>4. Error</h2>
      <p class="bloque__ayuda">En espanol y sin codigos. El usuario no tiene la culpa de que la red
falle.</p>
      <Estado
        tipo="error"
        titulo="No pudimos cargar las recetas"
        mensaje="Puede ser tu conexion o el servicio de recetas. Vuelve a intentarlo en unos
segundos."
```

```
      >
        <Boton>Reintentar</Boton>
      </Estado>
    </section>

    <aside class="aviso">
      <p class="aviso__titulo">Aviso tecnico para cuando lleguen los datos reales</p>
      <p>
        Cuando la API no encuentra resultados devuelve <code>meals: null</code>,
        no una lista vacia. Ese caso es el estado <em>Vacio</em>,
        no el de <em>Error</em>. Confundirlos es el fallo mas comun.
      </p>
    </aside>
  </div>
</Layout>

<style>
  .entrada {
    max-width: var(--ancho-lectura);
    color: var(--color-texto-suave);
  }

  .bloque {
    padding-top: var(--esp-5);
    margin-top: var(--esp-5);
    border-top: 1px solid var(--color-borde);
  }

  .bloque h2 {
    margin-bottom: var(--esp-1);
  }

  .bloque__ayuda {
    margin: 0 0 var(--esp-4);
    font-size: var(--texto-s);
    color: var(--color-texto-suave);
  }

  .aviso {
    margin-top: var(--esp-6);
    padding: var(--esp-4);
    background: var(--color-marca-suave);
    border: 1px solid var(--color-marca-borde);
    border-radius: var(--radio-m);
  }

  .aviso p {
    margin: 0;
  }

  .aviso__titulo {
    font-weight: 700;
    color: var(--color-marca-oscuro);
    margin-bottom: var(--esp-1) !important;
  }

  .aviso code {
```

```
    padding: 0.1rem 0.35rem;
    background: var(--color-fondo);
    border: 1px solid var(--color-borde);
    border-radius: 4px;
    font-size: 0.9em;
  }
</style>
```

> Qué decir: "Toda pantalla con datos existe en cuatro versiones. Si solo diseñamos la que tiene datos, diseñamos la cuarta parte. Esta página es nuestra referencia: cuando conectemos la API, cada estado ya existe y solo hay que decidir cuál mostrar."

Lee en voz alta el aviso técnico: cuando la API no encuentra nada devuelve `meals: null`. Eso es **vacío**, no **error**.

Paso 8 (120-130) · Probar, construir y publicar

1. `F12` → simulador → **320 px**. Recorre Inicio, Catálogo y Estados. En la consola, `true` en las tres.
2. Prueba con el teclado: `Tab` recorre las tarjetas y el favorito de cada una por separado. El foco se ve.
3. Construye y publica:

```
npm run build
git add .
git commit -m "feat: componentes del catalogo con datos de prueba y pagina de estados"
git push
```

Deben salir **6 páginas** y ningún archivo `.js` en `dist/`. Espera el check verde en **Actions** y abre la URL publicada en un celular.

Taller en parejas (130-170)

Enunciado: [[Taller 2026-11-13 - Componentes del catalogo]]. Mientras trabajan, pasa por los grupos con esta revisión de un minuto:

1. ¿Sus datos de prueba tienen los mismos campos que su API?
2. ¿La tarjeta recibe el objeto por props, o tiene los textos escritos adentro?
3. ¿La imagen tiene `width` y `height`?
4. A 320 px, ¿la consola responde `true` en el catálogo?

Cierre (170-180)

- Dos grupos muestran su catálogo publicado.
- Próxima clase: **datos reales desde la API**. Las tarjetas no cambian: solo cambia de dónde vienen los datos.

Errores frecuentes

Síntoma	Solución
`Cannot read properties of undefined (reading 'nombre')`	La tarjeta no recibió la receta. Revisa que se llame igual en los dos lados: `<Card receta={receta} />` y `const { receta } = Astro.props`
`recetas.map is not a function`	Se pasó otra cosa a `CardGrid`. Debe ser un arreglo: `recetas={recetas}`
`does not provide an export named 'recetas'`	El `import` usa llaves `{ recetas }` y el archivo debe tener `export const recetas`
Las imágenes no aparecen	Falta `${base}` en el `src`, o la ruta en los datos empieza con `/`
El clic en el favorito abre la receta	Falta `z-index: 2` en el botón (el `::after` del enlace queda encima)
La página se desplaza de lado a 320 px	Falta `min-width: 0` en la columna de resultados o en los filtros
El texto salta cuando cargan las imágenes	Faltan `width` y `height` en el `<img>`
Los filtros no filtran	Es lo esperado: todavía no hay JavaScript